IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

DIGITAL ELECTRONICS AND COMPUTER ARCHITECTURE

Tuesday, 6 May 2025, 2:30 PM

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible	First Marker(s) :	G.A. Constantinides, T. Clarke
	Second Marker(s) :	T. Clarke, G.A. Constantinides

© Imperial College London

THE QUESTIONS

1. a) Simplify the following Boolean expression. You may use any method, but the method must be clear from your answer.

$$A\,\overline{B} + \overline{\overline{A} + \overline{B} + C\overline{C}}$$

[4]

b) Simplify the Boolean expression $Y = \sum\{0,1,3,5,7,15\}$ using a Karnaugh map, leaving your answer in sum-of-products form. Show all your working.

[4]

c) Rewrite the Boolean expression $\overline{A\ (\overline{B}\,\overline{C} + B\,C)}$ in both minimal sum-of-products form and minimal product-of-sums form. Show all your working.

[4]

d) Write the decimal value -38_{10} (i) in 9-bit sign-and-magnitude with a binary encoding of magnitude, (ii) in 9-bit sign-and-magnitude with a BCD encoding of magnitude, (iii) in 9-bit one's complement (iv) in 9-bit two's complement.

[4]

e) Let $A(n)$ denote the stable Boolean value of the input A to the circuit in Figure 1.1 during the nth clock cycle of the circuit's operation. Let $X(n)$ denote the stable Boolean value of the output X during the nth clock cycle of the circuit's operation. Write a Boolean expression for $X(n+2)$ in terms of $A(n)$, $A(n+1)$ and $A(n+2)$.

[4]

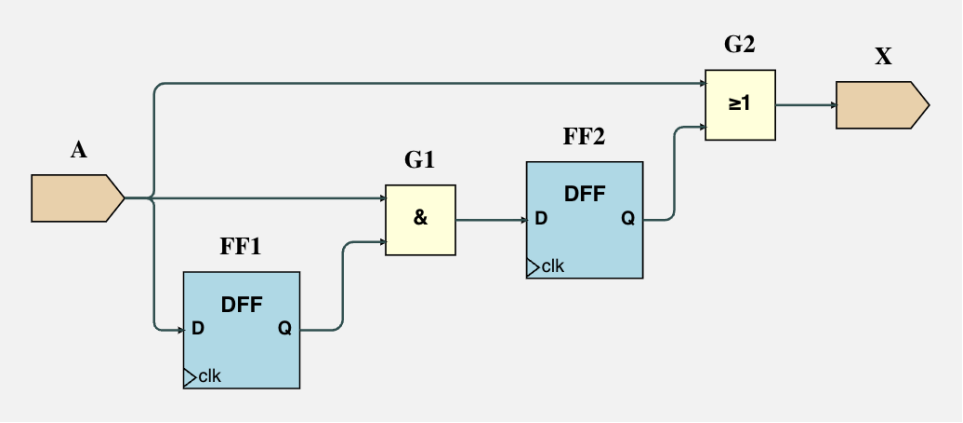


Figure 1.1 A sequential circuit.

f) Figure 1.2(a) shows an EEP1 program labelled `START` which is loaded into the EEP1 CPU instruction ROM starting at location 0. The CPU executes this program from location 0. Write the values, in hexadecimal, of the 8 registers R0 - R7 and the Program Counter, when the `JMP 0` instruction is executing.

[9]

g) Figure 1.2(b) shows an EEP1 program fragment labelled `JUMPS`. Some instructions are named by comments. For example the last instruction, `JMP 0`, is named `I6`. This code contains instructions that conditionally jumps over the instruction `XXi` for `i` $= 3,4,5,6$. The EEP1 Flags FlagC, FlagV, FlagN, FlagZ are designated C, V, N, Z.

i) State the name of the instruction that most recently wrote the value of each of the EEP1 Flags `N,Z,C,V` when instruction `XX4` executes.

[4]

ii) For each of the instructions `XX3, XX4, XX6`, write a Boolean expression of Flags `N,Z,C,V` that is true or false respectively when the instruction is or is not executed.

[3]

iii) Write four conditions on the signed or unsigned values of R0, R1, R2 which are true when R3, R4, R5, R6 respectively equals 1 at instruction `I6`. Your conditions should use Boolean algebra to combine equalities or inequalities on the signed or unsigned values of R0, R1, R2.

[4]

```
START MOV R0, 130
      MOV R1, #1
      SUB R1, #2
      ADC R1, #0
      MOV R2, #3
      ADC R2, R2, R2
      ADD R2, R2, R2
      ADD R2, R2, R2
      MOV R3, #21
      AND R3, #30
      MOV R4, #4
      LSL R4, R7, #14
      MOV R5, #5
      XSR R5, R5, #2
      JSR 1
      MOV R6, R7
      JSR 1
      SUB R6, R7, R6
      JMP 0
```

(a)

```
JUMPS  MOV R3, #0
       CMP R0, R1
       JGE J3
       MOV R3, #1     // XX3
J3     MOV R4, #0     // I1
       CMP R0, R1     // I2
       MOV R2, R2     // I3
       JCS J4         // I4
       JMI J4         // I5
       MOV R4, #1     // XX4
J4     MOV R5, #0
       SUB R7, R0, R1
       JLT J5
       SUB R7, R1, R2
       JGT J5
       MOV R5, #1     // XX5
J5     MOV R6, #0
       LSL R7, R2, #1
       JCC J6
       MOV R6, #1     // XX6
J6     JMP 0          // I6
```

(b)

Figure 1.2 Assembly Code

2. a) Write down the truth table of a full adder with input bits A, B and C_{in} and outputs S and C_{out}.

[2]

b) Hence or otherwise construct a minimal SOP implementation of the carry output of a full adder.

[5]

c) Figure 2.1 illustrates a circuit for bit-serial addition, where the component "Adder" denotes a full adder. Copy and complete the timing diagram shown in Figure 2.2 for the circuit shown in Figure 2.1.

[6]

The remainder of this question considers the hardware implementation of a radix-4 (*i.e.* base 4) adder with a standard digit set.

d) Let X and Y be radix-4 digits. State and explain the minimum and maximum possible values for $X + Y$.

[4]

e) By representing each radix-4 digit as two bits, construct a Boolean truth table for a single digit radix-4 adder, R4ADD. The adder needs to take two radix-4 input digits X and Y and a carry-in bit C_{in}, and produce a radix-4 sum digit S and a carry-out bit S_{out}.

[4]

f) Draw a modified version of the block diagram shown in Fig. 2.1 so that it performs radix-4 serial addition, calculating a radix-4 digit each clock cycle. Be careful to label the width (in bits) of any busses you use. You may keep R4ADD as a component within your design: you do not need to design the circuit implementing R4ADD.

[3]

g) Comment on the area and time efficiency of computing addition in radix-4 versus radix-2 using digit-serial arithmetic.

[6]

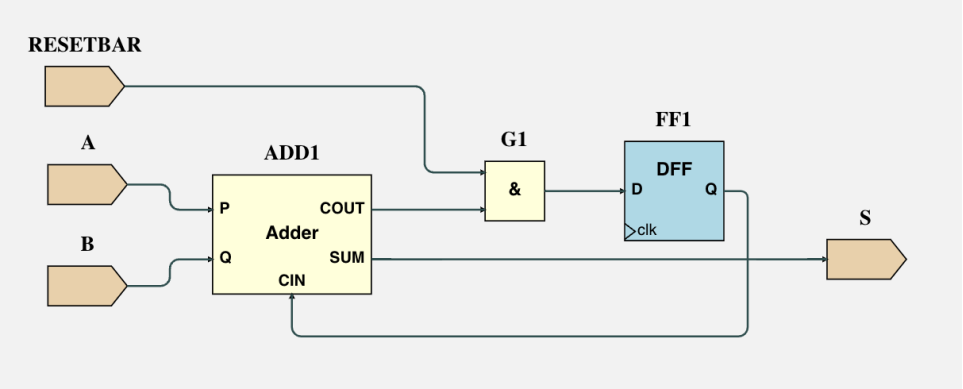


Figure 2.1 A bit-serial (*i.e.* radix-2 serial) adder.

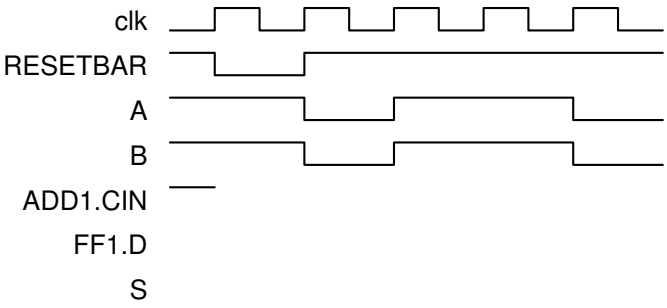


Figure 2.2 Timing diagram.

3. a) Figure 3.1 shows the logic of a CPU in four combinational blocks with the critical path delay of each block. The CPU without pipelining executes all blocks in a single clock cycle. You may assume delays through pipeline registers are zero. If EXEC1 and EXEC2 are placed in separate pipeline stages 10% of instructions executed have a 2-cycle pipeline stall. Otherwise there are no stalls. Using first 2 and then again 3 pipeline stages, show in each case where pipeline registers are placed to maximise instruction throughput and calculate that throughput.

[4]

b) State one advantage, and one disadvantage, of register offset over direct memory addressing. In each case give an example with explanation.

[4]

c) Figure 3.2 shows the contents of EEP1 data memory. Write an assignment using only R0 and mem[] notation that is equivalent to $x := y + z$.

[2]

d) Figure 3.2 shows the contents of EEP1 data memory. Write an EEP1 assembly program that will implement $x := y + z$ where x, y, z are as shown in Figure 3.2. Credit of up to 4 marks will be given to an answer that is compact as measured by the number of instructions.

[8]

e) The EEP1B is a 16 bit processor like EEP1 with new instructions `LDR, STR, LDRB, STRB` having operation defined in Figure 3.3. Byte addressing is little-endian. Write an EEP1B assembly program that implements $d8 := d8 + e8$ with $d8$, $e8$ as shown in Figure 3.2. Credit of up to 2 marks will be given to an answer that is compact as measured by the number of instructions.

[6]

f) $X(15:0)$, $Y(15:0)$ represent respectively *unsigned* and two's complement 16 bit *signed* integers. $S(15:0)$ is the SUM output of a binary adder with inputs X, Y.

i) Calculate, showing your working, the minimum and maximum possible value of $u(X) + z(Y)$.

[2]

ii) S is interpreted as *signed* therefore the condition for overflow from the addition of these two numbers is $V' = u(X) + z(Y) \neq z(S)$. Derive a simplified condition on the values of X, Y such that V' the same as the overflow from adding two signed numbers.

[2]

iii) Hence or otherwise determine a Boolean expression for V' in terms of $X(15), Y(15), S(15)$.

[2]

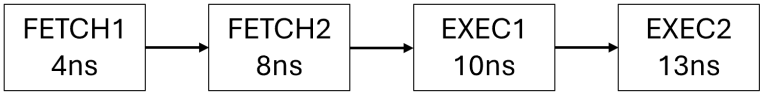


Figure 3.1 Combinational logic in a CPU

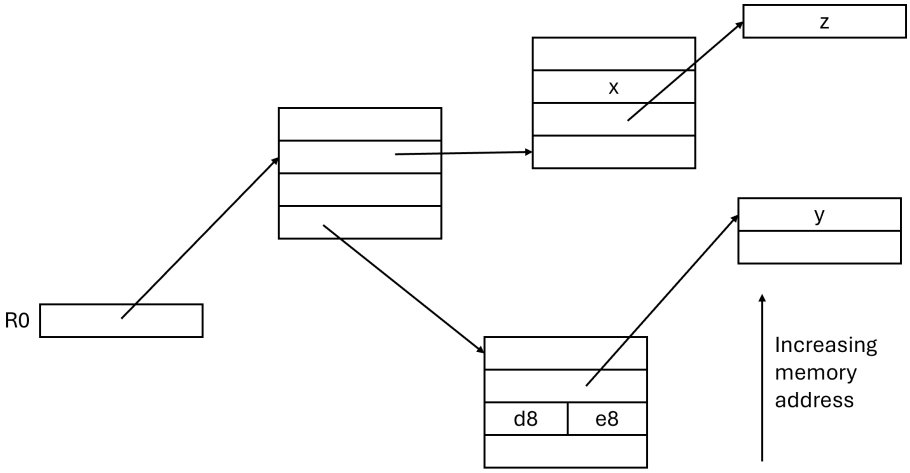


Figure 3.2 A diagram of data memory

Instruction	Operation (N is 8 bit signed)
`LDR Ra, [Rb,#N]`	Ra := mem_{16}[Rb + N]
`LDRB Ra, [Rb,#N]`	Ra := mem_{8}[Rb + N]
`STR Ra, [Rb,#N]`	mem_{16}[Rb + N] := Ra(7:0)
`STRB Ra, [Rb,#N]`	mem_{8}[Rb + N] := Ra(7:0)

Figure 3.3 EEP1B memory load/store instructions